Gemini OCR Service — Report

Gemini OCR Service — Khmer NID Card Text Detection & Recognition

Report scope: the Gemini service only — the component that reads a Cambodian National ID (NID) card image and returns its data as structured fields, ready to be hosted and integrated with the CheckinMe AI server. It also compares the two Gemini models that were tested for this job: **Gemini 2.5 Flash** and **Gemini 2.5 Pro**.

> This report intentionally covers *only* what lives in the Gemini service.

---

1. Executive summary

The CheckinMe app lets a user photograph their ID card. We need to turn that photo into clean, structured data — name, ID number, date of birth, gender, address, and so on — automatically.

Previously this was done with a separate OCR service (CamDX). This service **replaces that** with Google's **Gemini** AI vision model. A user uploads a raw card photo; the service sends it to Gemini, which both **reads** the text on the card (Khmer + English) and **organises** it into the exact fields the CheckinMe backend already expects.

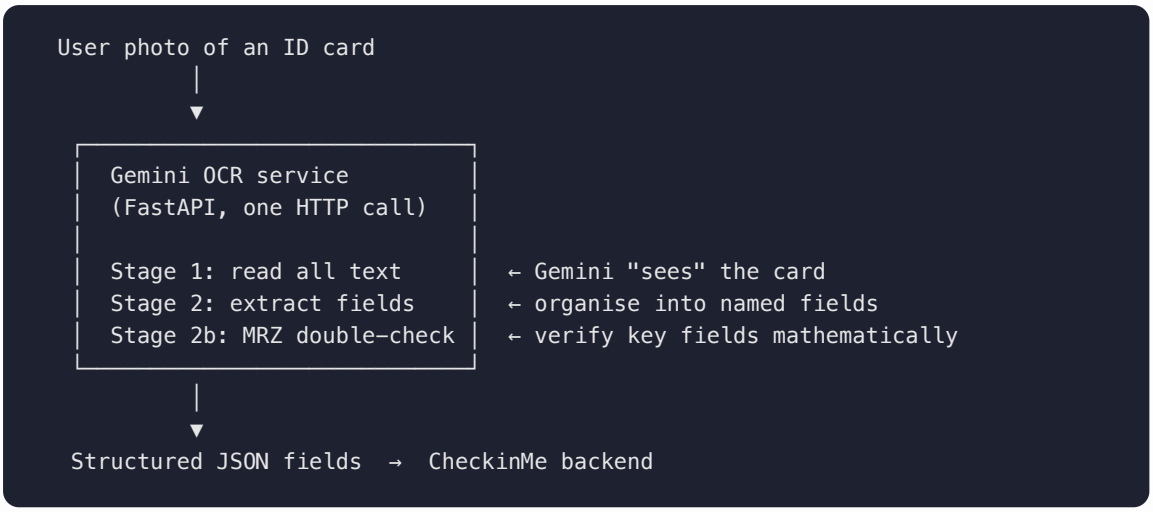
Two key points worth keeping in mind:

- **It is a drop-in replacement.** The data shape it returns is identical to the old service, so the rest of the CheckinMe system does not need to change.
- **It is "stateless."** It does not keep a database or remember anything between requests. One photo in, one set of fields out.

We tested two versions of the AI model — **Flash** (faster, cheaper) and **Pro** (slower, more accurate) — to decide which to run in production. The comparison and recommendation are in Section 6.

---

2. What the service does



The whole thing is **one call to Gemini** plus a small amount of local logic to verify the result. There is no local image cleanup on the production path — Gemini itself handles rotated, skewed, glare-affected, or low-resolution photos.

---

3. How it works

Stage 1 — Text detection & recognition

The raw image bytes are sent straight to Gemini together with a specialised prompt (`PROMPT` in `gemini/extractor.py`). The prompt tells Gemini:

- This is a Cambodian NID card with **both Khmer script and Latin/English** text.
- The photo may be rotated, skewed, low-res, or have glare/shadows — read it anyway.
- Exactly which fields the card contains, with the Khmer field labels for each (e.g. `កាលបរិច្ឆេទចេញ` = issue date), and how the three **MRZ** lines (the machine-readable zone at the bottom) are structured.

Gemini returns a `text` value: a full raw transcription of every visible character.

Stage 2 — Field extraction

The **same** Gemini call also returns a structured `fields` object. The prompt forces a strict contract:

- Return **only valid JSON**, no markdown, exactly the agreed keys.
- Use `null` for anything that can't be read with confidence — **never guess**.
- Dates as `DD/MM/YYYY` , gender exactly `M` / `F` , English names UPPERCASE, Khmer preserved verbatim (no transliteration).
- `temperature=0` and `response_mime_type="application/json"` are set on the API call so output is deterministic and machine-parseable.

Stage 2b — MRZ verification (the reliability trick)

The MRZ lines at the bottom of the card follow a fixed international format, so they can be parsed **mathematically** rather than trusting the AI's reading. `_parse_mrz()` decodes MRZ2 and MRZ3 to recover:

- `dob`, `gender`, `expiredDate` (from MRZ2)
- `lastNameEn`, `firstNameEn` (from MRZ3)

These deterministic values **backfill or override** the vision-read fields when they disagree. This is what makes the key identity fields trustworthy even if the visual OCR slips.

The output contract (identical to CamDX)

```
{
  "text":   "<full raw transcription>",
  "fields": {
    "idNumber":    "101325482",
    "lastNameKh":  "ជន",
    "firstNameKh": "ពេងហុង",
    "dob":         "17/01/2001",
    "gender":      "M",
    "lastNameEn":  "CHORN",
    "firstNameEn": "PENGHONG",
    "expiredDate": "18/01/2030",
    "issuedDate":  "18/01/2020",
    "address":     "ភូមិកាច់ត្រក ឃុំលាយបូរ ស្រុកត្រាំកក់ តាកែវ",
    "pob":         "ឃុំលាយបូរ ស្រុកត្រាំកក់ តាកែវ",
    "MRZ1":        "IDKHM1013254824<<<<<<<<<<<<<<<",
    "MRZ2":        "0101170M2610139KHM<<<<<<<<<<<8",
    "MRZ3":        "CHORN<<PENGHONG<<<<<<<<<<<<<<<"
  },
  "model":  "gemini-2.5-flash",
  "timing": { "total_ms": 0, "details": { "upload_read_ms": 0, "gemini_api_ms": 0
} }
}
```

The prompt — how Gemini is instructed

The service has no machine-learning model of its own to train. Its behaviour is controlled almost entirely by the **prompt** — the written set of instructions sent to Gemini alongside the image. The prompt *is* the configuration: it defines the task, the output shape, and the rules. The production path uses a single prompt, `PROMPT`, in `gemini/extractor.py`.

The prompt is structured in four parts:

1. **Role & context.** It tells Gemini it is "an OCR system specialised in Cambodian National ID cards," that the card carries **both Khmer script and Latin/English**, and that the photo may be rotated, skewed, low-resolution, or affected by glare, shadows, or background clutter — *"Read the card regardless."* This primes the model to handle messy real-world photos instead of expecting a clean scan.

2. **A description of the card's layout.** It enumerates every field the card contains and pairs each with its Khmer label so the model knows where to look — e.g. the 9-digit ID number at the top,

name in Khmer (`ឈ្មោះជាអក្សរខ្មែរ`), date of birth, gender, place of birth (`ទីកន្លែងកំណើត`), address (`អាសយដ្ឋានបច្ចុប្បន្ន`), issue date (`កាលបរិច្ឆេទចេញ`), expiry date (`កាលបរិច្ឆេទផុតកំណត់`), and the three **MRZ** lines — including the exact internal format of each MRZ line.

3. **The exact output schema.** It shows Gemini the precise JSON skeleton to return — a `text` field plus a `fields` object with exactly the agreed keys (`idNumber` , `lastNameKh` , `firstNameKh` , `dob` , `gender` , `lastNameEn` , `firstNameEn` , `expiredDate` , `issuedDate` , `address` , `pob` , `MRZ1` , `MRZ2` , `MRZ3`) — and *no extra keys, no markdown fences*. This is what keeps the output a true drop-in replacement for CamDX.

4. **Strict rules.** The closing rules remove ambiguity and prevent the model from inventing data:

   - Use `null` for anything that can't be read with confidence — **never guess**.
   - Dates must be `DD/MM/YYYY` exactly as printed.
   - `gender` must be exactly `"M"` or `"F"` .
   - English name fields must be UPPERCASE; Khmer must be preserved verbatim — **no transliteration**.
   - For MRZ lines, copy every character exactly, including all `<` padding; valid MRZ characters are only `A–Z` , `0–9` , and `<` .

The prompt works together with two API settings that make the output reliable: `temperature=0` (deterministic — the same card gives the same answer) and `response_mime_type="application/json"` (the model returns parseable JSON, not prose).

> **Other prompts in the file, not on the production path:** `PROMPT_WITH_REGIONS` (same task plus bounding-box coordinates for each field), `PROMPT_DETECT_CARD` (asks only for the card's four corners, used by the de-skew stage), and `PROMPT_V1` (an earlier draft, kept for reference). These belong to the disabled image-cleanup / annotation modes and can be promoted back if those features are revived.

Key code map

Symbol (`gemini/extractor.py`)	Responsibility
`PROMPT`	Detection + recognition + field-extraction instructions
`extract_from_bytes()`	**Production entrypoint** — Stage 1 + Stage 2 in one Gemini call
`extract_from_file()`	Same, from a file path (CLI/testing)
`_parse_mrz()`	Deterministic MRZ2/MRZ3 parse → field corrections
`_yymmdd_to_ddmmyyyy()`	MRZ date `YYMMDD` → `DD/MM/YYYY`
`_parse_json()`	Tolerant JSON parse (strips stray markdown fences)
`_sniff_mime()`	Detect JPEG/PNG/WebP from magic bytes
`_build_client()`	Build `google.genai.Client` from API key / env

> **Present but not on the production path** (kept for future use): card-corner detection + de-skew (`preprocess_card_from_bytes`), bounding-box annotation (`annotate_image` , `extract_with_regions_from_bytes`), the 2-call orchestrator (`process_card_from_bytes`), and the quality gate / image-save logic. Their HTTP routes are disabled. They can be promoted back if cleaned/annotated images are needed later.

---

4. The production endpoint

Method	Path	Gemini calls	Role
`POST`	`/gemini-ocr/upload`	1	**Production — CamDX replacement**
`GET`	`/health`	—	Liveness check

`POST /gemini-ocr/upload` takes a `multipart/form-data` upload: `file=<image>` , optional `model=<gemini-model>` . The Gemini call runs in a thread pool (`run_in_threadpool`) so the async server stays responsive while waiting on the API. Errors map cleanly to HTTP: `500` if the SDK is missing, `502` on a Gemini failure.

The disabled routes (`/upload/annotated` , `/upload/processed` , `/preview`) belong to the richer image-cleanup mode and are deliberately not registered for this phase.

---

5. Hosting & integration

Hosting (Google Cloud Run)

- Containerised via `Dockerfile` (Python 3.12-slim) and served by **gunicorn + uvicorn worker**: `--workers 1 --threads 8 --timeout 300` . One worker with threads is the right shape because the work is **I/O-bound** (waiting on the Gemini API), not CPU-bound.
- `--timeout 300` is deliberately larger than the Laravel caller's timeout so a slow **Pro** call doesn't get killed mid-request.
- `deploy/deploy.sh` deploys to Cloud Run from source. Defaults: region `asia-southeast1` (Singapore, closest to Cambodia), `1 CPU / 1Gi` , concurrency 8, scale-to-zero (`min-instances 0` , pay per request), max 5 instances.
- The **Gemini API key lives in Secret Manager**, never baked into the image; the model is passed as an env var (`GEMINI_MODEL`).

Integration with the CheckinMe AI server

The Laravel backend's `GeminiService::scanNid()` :

1. Stores the **original** uploaded image (no cleaned/annotated variants this phase).
2. Opens a `GeminiResult` audit row, `POST` s the image to `/gemini-ocr/upload` .

3. Records `status_code` + `response_data` (small — just `text`, `fields`, `model`, `timing`; no base64 image blobs).
4. Returns `$json['fields']`, which `GeminiPersonalInfoController::readIdCard()` maps to the personal-info response exactly as it did for CamDX.

Laravel config (`config/gemini.php` / `.env`):

```
GEMINI_SERVICE_BASE_URL=<cloud-run-url>
GEMINI_SERVICE_TIMEOUT=60
GEMINI_SERVICE_MODEL=gemini-2.5-flash
GEMINI_SERVICE_ENDPOINTS_SCAN_NID=/gemini-ocr/upload
```

---

6. Model comparison: Gemini 2.5 Flash vs 2.5 Pro

Both models run through the **exact same** code and prompt; only the model name changes (`GEMINI_MODEL`, or the `model` form field per request). So this is a clean apples-to-apples comparison of the model itself.

6.1 What each model is for

	Gemini 2.5 Flash	**Gemini 2.5 Pro**
Designed for	Speed & cost at scale	Maximum accuracy / hard reasoning
Latency	Lower (faster response)	Higher (noticeably slower)
Cost per call	Cheaper	More expensive
Best on	Clear, well-lit cards	Difficult cards: glare, skew, low-res, messy Khmer

6.2 Evidence already baked into the codebase

- **The deployment defaults to Flash.** `.env.example`, `deploy.sh` (`GEMINI_MODEL=gemini-2.5-flash`), and the architecture doc all default to Flash — i.e. Flash was chosen as the day-to-day production model.
- **The timeouts were tuned around Pro being slow.** Both the Docker `--timeout 300` and `deploy.sh` comments explicitly call out that the long timeout exists so that "slow gemini-2.5-pro calls" survive. This is direct evidence that **Pro is materially slower** in practice.
- **`config.py` currently sets `gemini_model = "gemini-2.5-pro"` as the in-code default**, while the deploy/env files set Flash. ⚠️ *This is an inconsistency worth resolving* — pick one default so local runs and deployed runs behave the same. (See Section 7.)

6.3 Measured results from your test

> Fill these in with the numbers you recorded. Use the same set of sample cards for both models (`sample_imgs/`) so the comparison is fair. `timing.gemini_api_ms` from the response gives

> you latency directly.

Metric	Gemini 2.5 Flash	Gemini 2.5 Pro
Avg. latency per card (`gemini_api_ms`)	_____ ms	_____ ms
Field accuracy — clear cards	_____ %	_____ %
Field accuracy — difficult cards (glare/skew/low-res)	_____ %	_____ %
Khmer name/address correctness	_____	_____
ID number / dates / gender correctness[1]	_____	_____
JSON well-formed (no parse fallback)	_____ / N	_____ / N
Relative cost per call	1× (baseline)	~ ___×

[1] Note: `dob` , `gender` , `expiredDate` , and English names are **MRZ-verified**, so both models should score near-identically on these whenever the MRZ is readable — the real differentiator is the Khmer fields and overall reading on hard images.

6.4 How to read the comparison

- **Flash** is the workhorse: fast, cheap, and good enough on the great majority of real submissions. For a user-facing flow where people wait for the result, the lower latency matters a lot.
- **Pro** earns its keep on the hard cases — poor lighting, heavy skew, faint or unusual Khmer text — where it tends to read more correctly, at the cost of speed and money.
- Because the **most sensitive fields are MRZ-verified in code**, the accuracy gap between the two narrows on exactly the fields that matter most for identity. That strengthens the case for Flash as the default.

6.5 Recommendation

- **Default to Gemini 2.5 Flash in production** (matches the existing deploy config): best balance of speed, cost, and accuracy for the typical card.
- **Keep Pro available as a fallback / escalation.** The `model` form field already lets the caller pick per request — so a card that fails the quality checks on Flash could be retried on Pro.
- **Resolve the default-model inconsistency** between `config.py` (Pro) and the deploy files (Flash) so behaviour is predictable everywhere.

---

7. Findings & recommendations

1. **Align the default model.** `app/core/config.py` defaults to `gemini-2.5-pro` , but `.env.example` and `deploy.sh` use `gemini-2.5-flash` . Make them consistent (recommend Flash) to avoid surprises between local and deployed runs.

2. **Record the measured numbers** from your Flash-vs-Pro test into the table in §6.3 so the recommendation is backed by data, not just defaults.
3. **Consider an automatic Flash→Pro escalation** for low-confidence results, using the existing per-request `model` override.
4. **Security:** the deploy currently allows unauthenticated access (`ALLOW_UNAUTH=true`) for dev simplicity. Before production, lock the Cloud Run service down (IAM / signed requests) since it processes personal ID data.

---

8. Glossary

- **OCR** — Optical Character Recognition: turning text in an image into machine text.
- **NID** — National ID card.
- **MRZ** — Machine-Readable Zone: the `<<<`-padded lines at the bottom of the card, designed to be read by machines and follow a fixed, verifiable format.
- **Stateless** — the service keeps no memory or database; each request is independent.
- **Latency** — how long the service takes to respond.
- **Cloud Run** — Google's service for running containers that scale automatically (including down to zero when idle, so you only pay per request).